

Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Sistemas de Bases de Dados
2º Trabalho de Laboratório

Turma: 51D

Nome: João Ramos Número: 50730 Mail: A50730@alunos.isel.pt

Nome: Fábio Pestana Número: 50756 Mail: A50756@alunos.isel.pt

Nome: Rafael Dias Número: 50773 Mail: A50773@alunos.isel.pt

Professor: António Teófilo

Data: 05 de Janeiro de 2025

Índice

Lista de Figuras 3
Introdução 4
1ª Parte do Projeto 5
1. Conceção 5
1.1 - Modelo entidade associação. 5
1.2 - Modelo relacional. 6
1.3 - Restrições de Integridade Aplicacional (RIA). 10
2. Concretização 11
2.1 Criação do modelo físico. 11
2.2 Carregamento de dados de teste. 12
2.3 Eliminação do modelo físico. 12
2ª Parte do Projeto 12
3. Alterações feitas na parte 1. 12
3.1 - Modelo entidade associação. 12
3.2 - Vistas adicionadas. 14
4. Java. 15
4.1 - Classe auxiliares. 15
4.2 - Administrador. 16
4.3 - Cliente. 17
4.4 - Condutor. 18
4.5 - Funcionário 19
4.6 - Gerente 21
4.7 - Web 21
Conclusão 23
Bibliografia 24

Lista de Figuras

Figura 1 - Modelo entidade-associação 5
Figura 2 - Novo modelo entidade-associação 13

Introdução

Este projeto, desenvolvido para a unidade curricular de Sistemas de Bases de Dados, foca-se numa empresa de aluguer de veículos automóveis, que oferece serviços de aluguer de veículos. A empresa opera em múltiplos locais, atendendo tanto clientes individuais como empresas.

O projeto encontra-se dividido em duas partes, na primeira parte, explorou-se os processos e metodologias envolvidas na criação de uma base de dados, desde a conceção de modelos conceptuais, lógicos e físicos até a implementação utilizando a linguagem *Structured Query Language* (SQL). Na segunda parte, desenvolveu-se uma aplicação Web que funciona como uma interface com acesso à base de dados criada na primeira parte do projeto. Para alcançar este propósito, utilizaremos as linguagens *Structured Query Language* (SQL) e *Java Database Connectivity* (JDBC), bem como *JavaServer Pages* (JSPs) e *Servlets*, permitindo adicionar, modificar e consultar os dados presentes na base de dados.

1ª Parte do Projeto

1. Conceção

1.1 - Modelo entidade associação.

Na figura seguinte, encontra-se o desenho do modelo entidade associação.

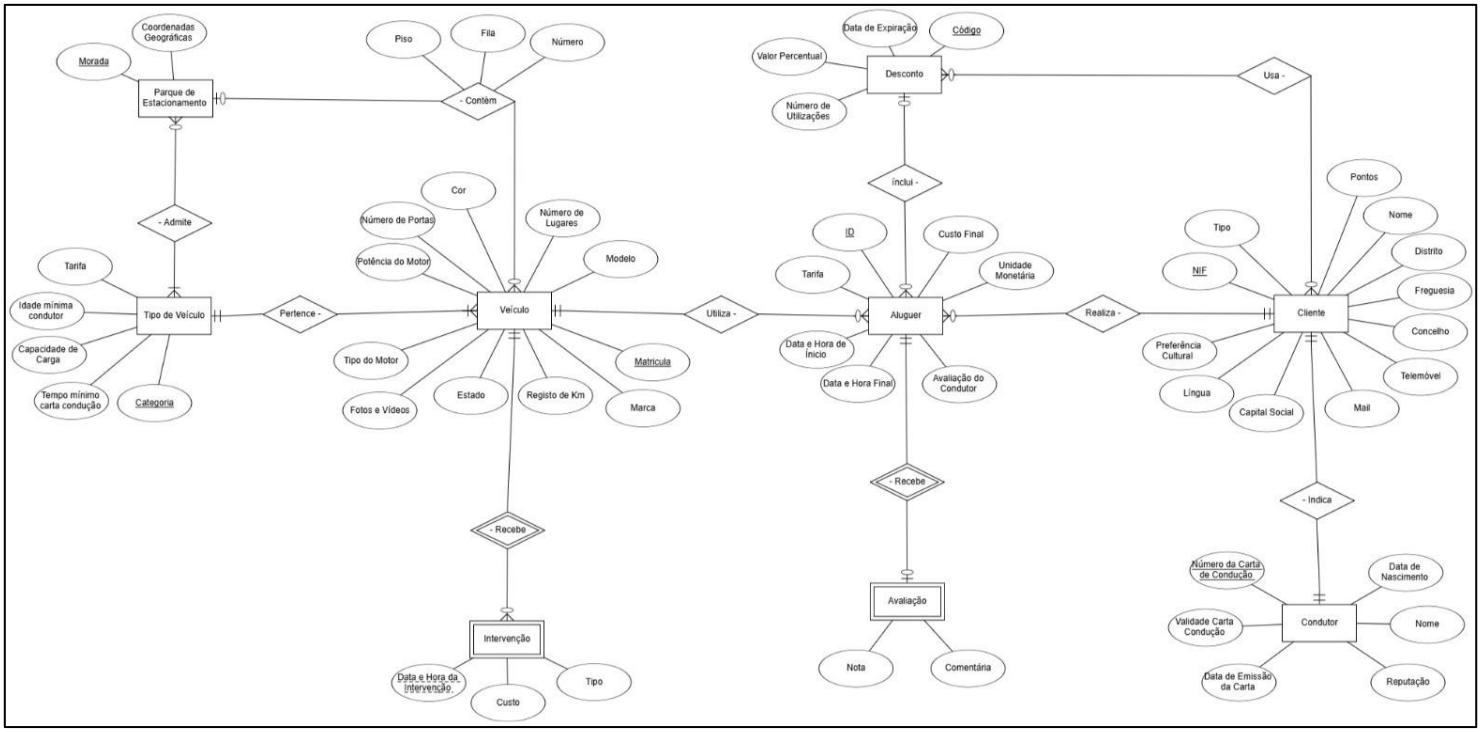


Figura 1 - Modelo entidade-associação

Na elaboração do modelo entidade associação, seguiu-se os seguintes pressupostos:

Parque de Estacionamento - A morada foi escolhida como chave primária pela possibilidade de existir dois ou mais parques muito próximos, o que pode acabar por causar problemas caso as coordenadas geográficas sejam escolhidas com pouca exatidão.

Tipo de Veículo - Foi pressuposto que a empresa pode comprar veículos pertencentes a um tipo que ainda nenhum dos seus parques aceite.

Intervenção - Foi pressuposto que os únicos tipos de intervenções que um veículo pode receber são reparações e manutenções.

Aluguer - Foi criado um atributo "ID" de modo a melhor identificar o aluguer. O atributo "Avaliação do Condutor" foi criado para que os condutores tenham a si atrelados uma reputação.

Cliente - O atributo "Pontos" foi criado de modo a fazer com que um cliente possa receber pontos ao realizar alugueres. Esses pontos podem ser gastos de modo a criar um desconto próprio de uso único.

Condutor - O atributo "Reputação" é uma média das avaliações que o condutor recebe pela empresa. As avaliações terão em conta o estado com que o veículo foi entregue à empresa depois de um aluguer.

1.2 - Modelo relacional.

O modelo relacional, apresenta a estrutura de dados para o sistema de uma empresa de aluguer de veículos, especificando as entidades, domínios dos atributos e os tipos de chaves (primárias, candidatas e estrangeiras). Cada entidade representa um aspeto fundamental do negócio, como veículos, clientes, parques de estacionamento, entre outros.

De seguida encontram-se as várias entidades criadas no modelo entidade-associação.

Parque de estacionamento (morada, coordenadas geográficas).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- A morada é um `VARCHAR(150)`, ou seja, uma `string` com 150 caracteres.
- As coordenadas geográficas é um `VARCHAR(25)`, ou seja, uma `string` com 25 caracteres. Para assim registar a latitude e longitude com uma certa precisão.

Chave Primária: {morada}

Chaves Candidatas: {morada, coordenadas geográficas}

Tipo de veículo (categoria, tempo mínimo de carta de condução, capacidade de carga, idade mínima do condutor, tarifa).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- A categoria é um `VARCHAR(30)`, que só aceita os seguintes valores: “familiar“, “comercial“, “motociclo“, “transporte de mercadorias“, “desportivo“.
- O tempo mínimo de carta de condução, a capacidade de carga, a idade mínima do condutor e a tarifa são números inteiros. O tempo mínimo da carta e idade mínima do condutor têm como unidade: anos. A capacidade de carga tem como unidade: quilogramas. A tarifa tem como unidade: euros/dia.

Chave Primária: {categoria}

Chaves Candidatas: {categoria, tarifa}

Veículo (matrícula, cor, marca, modelo, número de lugares, número de portas, potência do motor, tipo do motor, estado, fotos/vídeos, registo de km, categoria).

Domínio dos atributos:

- O único atributo que aceita `null` é o das fotos/vídeos, pois considera-se que não é obrigatório colocar fotos e vídeos do veículo.
- A matrícula é um `VARCHAR(10)`, ou seja, uma `string` com 10 caracteres.
- A cor e o modelo são um `VARCHAR(20)`, ou seja, uma `string` com 20 caracteres

- A marca é um `VARCHAR(25)`, ou seja, uma `string` com 25 caracteres.
- O número de lugares, número de portas e a potência do motor são números inteiros.
- O tipo do motor é um `VARCHAR(15)`, que só aceita os seguintes valores: “gasolina”, “gasóleo”, “GPL” e “elétrico, híbrido”.
- O estado é um `VARCHAR(15)`, que só aceita os seguintes valores: “disponível”, “alugado” e “manutenção”.
- Fotos/vídeos é um `Blob`, ou seja, um valor binário de alta dimensão capaz de guardar informação sobre estes tipos de media.
- O registo de km é um número com casas decimais, representado por um `float`.

Chave Primária e Candidata: {matrícula}

Chave Estrangeira: {categoria}

Intervenção (data e hora da intervenção, custo, tipo, matrícula).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- A data e hora da intervenção é um `DateTime`, ou seja, guarda informações sobre a data e hora da execução da intervenção no veículo.
- O custo é um número com casas decimais, representado por um `float`.
- O tipo é um `VARCHAR(20)`, que só aceita os seguintes valores: “reparação” e “manutenção”.

Chave Candidatas: {data e hora da intervenção, matrícula}

Chave Estrangeira: {matrícula}

Aluguer (ID, tarifa, unidade monetária, data e hora de início, data e hora final, custo final, avaliação do condutor, matrícula, NIF).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- O ID é um `VARCHAR(10)`, ou seja, uma `string` com 10 caracteres.
- A tarifa é um número inteiro.
- A unidade monetária é um `VARCHAR(10)`, que só aceita os valores de uma lista de unidades monetárias. Lista {'EUR', 'USD', 'GBP', 'BRL', 'JPY', 'CHF','CNY}
- A data e hora de início e data e hora final são ambos um `DateTime`, para assim conseguirem guardar informações sobre a data e a hora.
- O custo final é um número com casas decimais, representado por um `float`.
- A avaliação do condutor é um inteiro, com valores entre 1 e 5, sendo 5 a indicação que o veículo foi entregue nas mesmas condições do seu levantamento. O valor 1 indica que o veículo foi entregue com danos ou em péssimas condições.

Chave Primária: {ID}

Chave Candidatas: {ID, data e hora de início, data e hora final}
Chave Estrangeira: {matrícula} - Veículo; {NIF} - Cliente

Desconto (código, número de utilizações, data de expiração, valor percentual).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- O código é um `VARCHAR(12)`, ou seja, uma `string` com 12 caracteres.
- O número de utilizações é um número inteiro. Caso o desconto tenha uso ilimitado durante um período, o valor guardado na base de dados deve ser -1.
- A data de expiração é um `Date`, ou seja, guarda informações sobre a data.
- O valor percentual é um número inteiro.

Chave Primária e Canditada: {código}

Avaliação (ID, nota, comentário).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- A nota é um *enumerator* (`Enum`), com valores que representam a avaliação feita pelo cliente ao escolher entre três valores possíveis: “Não vou voltar”, “Gostei” e “Adorei”.
- O comentário é um `VARCHAR(500)`, ou seja, uma `string` com 500 caracteres, optou-se por um `VARCHAR` em fez de um `TEXT`, para assim limitar o tamanho do texto.

Chave Candidatas: {comentário, ID}
Chave Estrangeira: {ID}

Cliente (NIF, tipo, nome, distrito, concelho, freguesia, mail, telemóvel, capital social, língua, preferência cultural, pontos).

Domínio dos atributos:

- Caso a morada do cliente não seja portuguesa, os atributos Distrito, Concelho e Freguesia podem ter valores `null`.
- Se o cliente for uma pessoa individual, o atributo capital social pode ter valor de `null`.
- A preferência cultural pode ter valor `null`, os restantes atributos não podem ter valores a `null`.
- O NIF é um `VARCHAR(12)`, ou seja, uma `string` com 12 caracteres.
- O tipo é um `VARCHAR(10)`, que só aceita os valores: "empresa" e "pessoa".
- O nome, concelho e a freguesia são `VARCHAR(35)`, ou seja, `strings` com 35 caracteres.
- O distrito e a língua são `VARCHAR(20)`, ou seja, `strings` com 20 caracteres.
- O mail é um `VARCHAR(70)`, ou seja, uma `string` com 70 caracteres.

- O telemóvel é um `VARCHAR(15)`, ou seja, uma `string` com 15 caracteres.
- O capital Social é um número inteiro.
- A preferência cultural é um `VARCHAR(50)`, ou seja, uma `string` com 50 caracteres.
- Os pontos é um número inteiro, que indica os pontos acumulados do cliente (podem ser usados em descontos).

Chave Primária: {NIF}

Chave Candidatas: {NIF, capital social, mail, telemóvel}

De seguida encontram-se as várias associações entre entidades do modelo entidade-associação.

Parque de estacionamento_Veículo (morada, piso, fila, número, matrícula).

Cada parque de estacionamento pode conter vários veículos, mas também pode não conter nenhum (caso não haja veículos estacionados no momento). Cada veículo está associado a um parque de estacionamento, mas essa associação não é obrigatória, pois o veículo pode estar alugado e, portanto, fora de qualquer parque. Caso esteja no parque, ele poderá ser encontrado no piso x, fila y, no número z.

Chave Candidata: {matrícula}

Chaves estrangeiras: {matrícula} e {morada}

Parque de estacionamento_Tipo de veículo (morada, categoria).

Cada parque de estacionamento tem de admitir pelo menos um tipo de veículos. Cada tipo de veículo pode ser admitido em vários parques, mas também pode não estar associado a nenhum parque (a empresa pode obter um tipo de veículo novo sendo que este ainda não é aceite por nenhum parque).

Chave Candidata: {morada, categoria}

Chaves estrangeiras: {morada} e {categoria}

Desconto _Aluguer (ID, código).

Um aluguer pode utilizar um desconto, mas como não é necessário ter um desconto para realizar um aluguer, este tem de ser opcional. No entanto, um desconto pode ser utilizado em vários diferentes alugueres (caso de descontos para vários clientes, como descontos de Natal).

Chave Candidata: {ID}

Chaves estrangeiras: {ID} e {código}

Cliente_Condutor (NIF, número da carta de condução, nome, data de emissão da carta, data de nascimento, validade da carta de condução, reputação).

Cada cliente deve obrigatoriamente indicar um condutor para o aluguer, que pode ser ele próprio ou outra

pessoa. A indicação do condutor é essencial, pois o sistema precisa de registar quem será o responsável pela condução do veículo.

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- O número da carta de condução é um VARCHAR(12), ou seja, uma string com 12 caracteres.
- O nome é um `VARCHAR(35)`, ou seja, uma `string` com 35 caracteres.
- A data de emissão da carta, a data de nascimento e a validade da carta de condução são do tipo `Date`, ou seja, guardam informações sobre as datas.
- A reputação é um número com casas decimais, representado por um `float`.

Chave Candidatas: {NIF, número da carta de condução}

Chaves estrangeiras: {NIF}, como o cliente tem mais associações com outras entidades, ele é uma entidade própria logo a chave estrangeira é a primária do cliente.

Cliente_Desconto (NIF, código).

Cada cliente pode utilizar vários descontos ao longo do tempo, mas também pode não utilizar nenhum. Por outro lado, cada desconto pode ser utilizado por vários clientes diferentes, pois as promoções geralmente estão disponíveis para vários utilizadores. Como um cliente pode optar por não usar nenhum desconto ou um desconto pode não ser utilizado a associação é opcional para ambas as entidades.

Chave Candidata: {NIF, código}

Chaves estrangeiras: {NIF} e {código}

1.3 - Restrições de Integridade Aplicacional (RIA).

As Restrições de Integridade Aplicacional (RIA) representam as regras e procedimentos necessários que não conseguem ser apresentados nos modelos anteriores, logo estas regras são implementadas na aplicação. Abaixo encontram-se as restrições usadas no projeto.

Cálculo do Custo Final do Aluguer: O custo final do aluguer é determinado com base nas tarifas aplicáveis, considerando a duração do aluguer e o tipo de veículo. Caso o cliente tenha direito a um desconto válido, este é aplicado sobre o custo calculado. O custo final também deve ser apresentado segundo a unidade monetária escolhida.

Adição de Fotos e Vídeos (Conversão): A adição de fotos e vídeos à base de dados será pela aplicação, visto que tanto fotos quanto vídeos precisam ser previamente convertidos para binário ou Base64.

Mudança da Nota (Avaliação) para um Enum: A avaliação é feita pelo cliente ao escolher entre três campos: “Não vou voltar”, “Gostei”, “Adorei”. A aplicação terá de converter essa escolha para um `Enum` a ser guardado na

base de dados.

Geração de um ID para o Aluguer: Para cada novo aluguer, um identificador único (ID) deve ser gerado automaticamente. Este ID é essencial para identificar cada aluguer e avaliação no sistema.

Um aluguer para ser válido tem de estar no intervalo (1 hora a 14 dias): A diferença entre a data inicial e final do aluguer deve encontrar-se no intervalo mencionado.

Cálculo da reputação do condutor: A reputação do condutor será calculada pela média de avaliações recebidas pelo condutor em cada um dos alugueres que ele participou.

Colocar dados dos Clientes a `null`: Caso um cliente da empresa seja uma pessoa, esta não necessita de indicar a sua capital social (dado fica a `null`). Caso o cliente não tiver uma morada portuguesa, as informações sobre distrito, concelho e freguesia não serão preenchidas, isto é, serão colocadas a `null`.

2. Concretização

2.1 Criação do modelo físico.

Logo após a conceção do modelo entidade associação, modelo relacional e a definição das restrições de integridade aplicacional, foi iniciada a concretização da base de dados a partir de um *script* SQL DDL.

Primeiramente foi criada uma base de dados chamada **SBD_TP1** utilizando o comando `CREATE DATABASE SBD_TP1`, esta base de dados irá conter as tabelas das entidades e associações presentes nos modelos previamente apresentados. Usando a base de dados SBD_TP1 (`USE SBD_TP1`) foram criadas várias tabelas com o comando `CREATE TABLE` para cada uma das entidades e associações entre elas. Cada tabela contem diversas colunas que correspondem aos atributos pertencentes às entidades, indicando o seu tipo (Ex: VARCHAR(x), INT, BLOB, FLOAT), a possibilidade do atributo não conter nenhum valor (`Null`) e se o atributo é uma chave primária (*Primary Key*).

Caso um atributo possa apenas fazer parte de uma lista de valores podemos usar a função `CHECK IN`, em que "CHECK" seleciona o atributo a ser limitado e "IN" indica a lista de valores possíveis.

Após todas as tabelas terem sido criadas, foi indicado para cada uma as suas chaves estrangeiras (segundo o modelo relacional referido anteriormente). Isto considera-se uma boa prática, visto que todas as tabelas já estão criadas antes da adição das suas chaves estrangeiras, o que evita erros no que diz respeito à ordem em que as tabelas são criadas.

Por fim, criou-se quatro vistas, ou seja, criaram-se estruturas com o objetivo de simplificar o acesso a dados e facilitar consultas. As vistas criadas foram as seguintes:

- **AlugueresDetalhados**: Esta vista consolida informações detalhadas sobre os alugueres realizados. Ela reúne dados dos clientes, condutores e veículos, permitindo uma visualização completa de cada transação.

- **HistoricoDasIntervencoes**: Oferece o histórico das intervenções realizadas nos veículos, incluindo o tipo de intervenção (manutenção ou reparação), a data e o custo.
- **ResumoDosVeiculos**: Resume informações importantes sobre cada veículo, como o número total de alugueres realizados e a receita gerada.
- **InterpretacaoDosDescontos**: Esta vista traduz os descontos que não têm valores de utilizações, substituindo o seu valor de -1 pelo termo "Ilimitado". Isto facilita a interpretação por parte dos utilizadores e melhora a legibilidade dos dados.

2.2 Carregamento de dados de teste.

Seguidamente à criação do modelo físico foi criado um *script* SQL DML que tem como objetivo inserir dados nas tabelas criadas previamente, utilizando o comando `INSERT INTO VALUES`, em que "INSERT INTO" seleciona a tabela em que os dados serão inseridos e "VALUES" indica os valores que serão inseridos.

2.3 Eliminação do modelo físico.

Foi também criado um *script* SQL DDL para eliminar o modelo físico da base de dados. Para tal utilizamos a função `ALTER TABLE` que permite alterar uma tabela já criada, depois utilizamos a função `DROP CONSTRAINT` que permite a remoção das chaves estrangeiras das tabelas, com isto removemos as referências que uma tabela tem em relação com uma outra, impedindo assim erros posteriores na remoção de tabelas. De seguida, utilizamos a função `DROP TABLE IF EXISTS` que remove uma tabela da base de dados se esta existir. Por fim, utilizamos a função `DROP DATABASE IF EXISTS` que apaga a base de dados criada no primeiro ponto, se esta existir.

2ª Parte do Projeto

3. Alterações feitas na parte 1.

3.1 - Modelo entidade associação.

Na figura seguinte, encontra-se o desenho do novo modelo entidade associação.

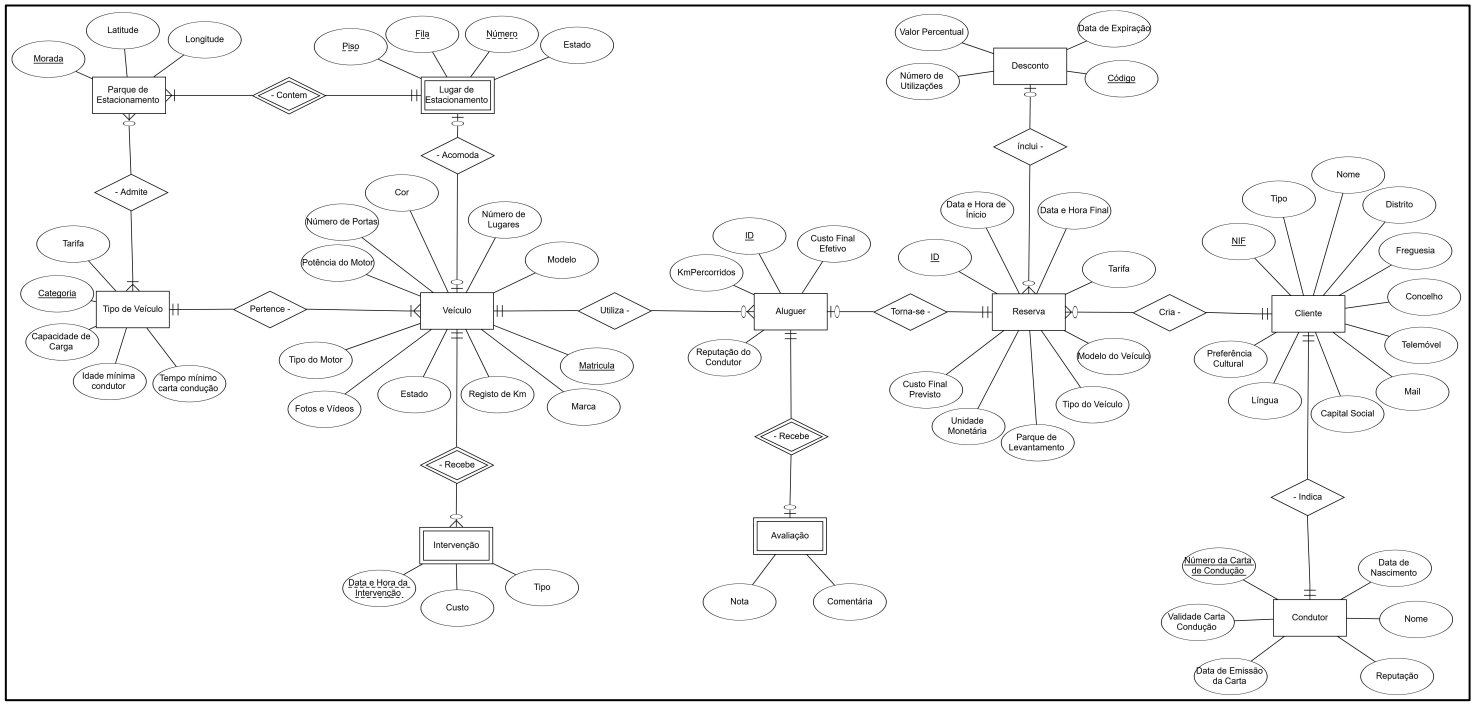


Figura 2 - Novo modelo entidade-associação

Devido aos novos requisitos, foi necessário alterar o modelo entidade associação. Foi criada a entidade Lugar de Estacionamento, que é uma entidade fraca em ralação à entidade Parque de Estacionamento. Esta nova entidade foi introduzida com o objetivo de melhor identificar quais lugares de estacionamento estão vagos e quais já se encontram ocupados por veículos.

Lugar de Estacionamento (Piso, Fila, Número, Estado, Morada).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- O Piso, Fila e Numero são números inteiros.
- O Estado é representado por um `VARCHAR(15)`, que só aceita os valores “vago” e “ocupado”.

Chaves Candidatas: {Piso, Fila, Numero, Morada}

Chave Estrangeira: {Morada}

Foi também criada uma entidade Reserva para refletir a nova funcionalidade que permite aos clientes reservar veículos. Com esta alteração, a antiga entidade Aluguer foi dividida em duas: **Aluguer** e **Reserva**, permitindo uma distinção clara entre reservas (alugueres que ainda não foram validados por um funcionário) e alugueres (alugueres validados, a decorrer ou já concluídos). Uma Reserva torna-se um Aluguer quando um funcionário atribui um veículo à mesma. Um Aluguer dá-se como terminado quando um condutor entrega o veículo num parque e um funcionário dá o aluguer como concluído. Além disso, o atributo Desconto, anteriormente associado ao Aluguer, passou a estar relacionado com a entidade Reserva, dado ao facto do cliente utilizar um desconto quando reserva um veículo.

Reserva (ID, Tarifa, UnidadeMonetaria, DataHoraInicio, DataHoraFinal, CustoFinalPrevisto, NIF, TipoVeiculo, Modelo, ParqueEstacionamento).

Domínio dos atributos:

- Apenas TipoVeiculo e Modelo podem ser `null`.
- O ID é um `VARCHAR(10)`.
- A Tarifa é um número inteiro.
- A UnidadeMonetaria é um `VARCHAR(10)` que só aceita os valores 'EUR', 'USD', 'GBP', 'BRL', 'JPY', 'CHF', 'CNY'.
- DataHoraInicio e DataHoraFinal são do tipo `DateTime`. DataHoraFinal deve ser maior que DataHoraInicio.
- O CustoFinalPrevisto é um número com casas decimais, representado por um `float`.
- NIF é um `VARCHAR(12)`.
- TipoVeiculo é um `VARCHAR(30)`.
- Modelo é um `VARCHAR(20)`.
- ParqueEstacionamento é um `VARCHAR(150)`

Chaves Candidatas: {ID}

3.2 - Vistas adicionadas.

Para facilitar as consultas para a aplicação Web, o grupo desenvolveu as seguintes vistas:

- **AlugueresEIntervencoes**: Esta vista consolida informações de alugueres e intervenções de veículos. Ela exibe informações sobre a matrícula, marca, modelo, tipo de intervenção, em serviço (significa que o veículo foi alugado) ou manutenção/reparação (intervenção feita no veículo), e os custos envolvidos. No caso de alugueres, exibe também o ID do aluguer e as datas de início e fim.
- **LucroTotalPorMarca**: Calcula o lucro total (em euros) gerado por cada marca de veículo, considerando as tarifas de aluguer, convertidas para euros com base na unidade monetária utilizada.
- **EstadoReservas**: Categoriza o estado atual das reservas, indicando se estão pendentes, validadas ou passadas (se a data final já expirou). Também exibe o custo previsto, o custo efetivo (se houver aluguer associado), e informações do veículo associado à reserva.
- **DetalhesAlugueresCondutor**: Apresenta informações sobre os alugueres e os condutores associados, incluindo o nome e a reputação do condutor, o custo previsto da reserva, e a matrícula do veículo alugado.
- **AlugueresIncompletos**: Foca em alugueres que ainda estão incompletos, apresenta os detalhes do cliente, veículo (marca, modelo, cor, matrícula), e a localização do veículo no estacionamento. Serve para identificar alugueres que precisam ser finalizados.
- **ReservasComAluguer**: Apresenta as reservas que já têm um aluguer associado, mostrando detalhes completos de ambas as entidades, como o custo previsto, o custo efetivo, avaliações do condutor, e quilômetros percorridos.

4. Java.

4.1 - Classe auxiliares.

Para o desenvolvimento da aplicação, inicialmente criou-se uma classe `Stub` com o propósito de estabelecer uma conexão entre a aplicação e a base de dados MySQL. O objetivo principal desta classe é fornecer métodos para a execução de comandos SQL, incluindo consultas e alterações na estrutura ou nos dados.
A classe `Stub` é constituída pelos métodos:

- **Stub**: Construtor da classe e cria uma conexão com a base de dados MySQL
- **getConnection**: Estabelece uma conexão com a base de dados.
- **sucessoInsert**: Verifica se um comando de update, insert ou delete afetou pelo menos uma linha na base de dados.
- **executeInsertQuery**: Executa comandos SQL para atualizar, inserir ou apagar informações na base de dados segundo a *query* SQL que ele recebe e os seus parâmetros.
- **closeConnection**: Fecha a conexão com a base de dados.
- **printResults**: Exibe na consola os resultados de uma consulta SQL.
- **executeQuery**: Executa consultas SQL e retorna os resultados num array bidimensional, `String[][]` .
- **getResults**: Processa os resultados de uma consulta SQL e retorna um array bidimensional com os resultados da consulta.
- **gerarId**: Gera um identificador único composto por um prefixo e um número aleatório de 5 dígitos (utilizado para a criação de alugueres e reservas).

Posteriormente, criamos uma classe `Auxiliar` que fornece vários métodos para validação, formatação de dados, interação com a base de dados e geração elementos HTML. Esta classe será usada como suporte para diversas funcionalidades do sistema.
A classe `Auxiliar` é constituída pelos métodos:

- **isNameValid**: Verifica se um nome é válido. Um nome é considerado válido se contiver apenas letras ou espaços.
- **isNumberValid**: Verifica se um número é válido. O número deve conter apenas dígitos, ter no mínimo 8 e no máximo 15 caracteres.
- **isValidEmailAddress**: Valida se um e-mail está no formato correto.
- **normalizaTexto**: Normaliza o texto fornecido, capitalizando a primeira letra de cada palavra e convertendo o restante para minúsculas.
- **verificarMatricula**: Verifica se uma matrícula já existe na base de dados (através da tabela `Veiculo`).
- **verificarMatriculaDisponivel**: Verifica se uma matrícula existe e está disponível.
- **verificarNIF**: Verifica se um NIF já existe na base de dados (através na tabela `Cliente`).
- **verificarCartaConducao**: Verifica se um número de carta de condução existe na tabela `Cliente_Condutor`.

- **listarParques**: Lista os parques de estacionamento registados, retornando apenas o endereço.
- **listarParqueWeb**: Similar ao `listarParques()`, mas configurado para funcionar na Web.
- **listarParquesDadosWeb**: Retorna informações completas dos parques, incluindo a morada, latitude e longitude.
- **tiposVeiculos**: Retorna a lista de categorias de veículos.
- **gerarIdReserva**: Gera um ID único para reservas. Caso o ID gerado já exista, gera outro até encontrar um ID único.
- **idReservaExiste**: Verifica se um ID de uma reserva já existe.
- **idAluguerExiste**: Verifica se um ID de um aluguer existe.
- **generateHtmlTable**: Gera uma tabela HTML a partir de dados fornecidos.
- **generateHtmlTableWithRank**: Similar a `generateHtmlTable()`, mas inclui uma coluna de rank (posição).
- **gerarFichaClienteHTML**: Gera um relatório em HTML sobre um cliente, incluindo informações como nome, NIF, preferências culturais e, se aplicável, capital social.
- **gerarOpcoes**: Gera as opções de um `select` HTML, segundo um indice dos resultados de uma consulta SQL.

4.2 - Administrador.

A partir deste ponto, criámos classes que se responsabilizam pela resolução de todos os requisitos dos seus respetivos utilizadores.

Começaremos pela classe `Administrador`, responsável por todas as funcionalidades referentes ao administrador. A classe `Administrador` é constituída pelos métodos:

- **atualizarCliente**: Método para responder a parte do primeiro ponto do perfil Administrador, este atualiza um atributo de um cliente na base de dados, validando o atributo e o valor. Para tal, após a validação do atributo e do seu valor, faz-se uma *query* SQL onde atualiza-se o valor do atributo, através do NIF do cliente.
- **atualizarCondutor**: Método para responder a parte do primeiro ponto do perfil Administrador, este atualiza um atributo de um condutor na base de dados, validando o atributo e o valor. Para tal, após a validação do atributo e do seu valor, faz-se uma *query* SQL onde atualiza-se o valor do atributo, através do NIF e número de carta de condução (**cc**) associados ao condutor.
- **criarCliente**: Método para responder a parte do primeiro ponto do perfil Administrador, este cria um cliente na base de dados, verificando o NIF (se é válido e verifica se este já existe na base de dados), email, número de telefone e nome, após a validação desses valores, faz-se uma *query* SQL para inserir o novo cliente na base de dados.
- **criarCondutor**: Último método para responder ao primeiro ponto do perfil Administrador, este cria um condutor associado a um cliente, validando o NIF, o número da carta de condução (verifica se já existe na base de dados o conjunto do NIF e carta de condução e valida se cada elemento separadamente) e o nome, após a validação desses valores, faz-se uma *query* SQL para inserir o novo condutor na base de dados.

- **criarCondutorWeb**: Semelhante ao método **criarCondutor()**, mas para a Web.
- **criarVeiculo**: Método para responder a parte do segundo ponto do perfil Administrador, este cria um veículo na base de dados, realizando verificações de valores válidos para os parâmetros fornecidos (verifica se já existe um veículo com a matrícula do veículo). Após a validação dos valores, faz-se uma *query* SQL onde se insere uma nova entrada na tabela Veiculo.
- **atualizarVeiculo**: Último método para responder ao segundo ponto do perfil Administrador, este atualiza um atributo de um veículo na base de dados, validando o atributo, o valor e a matrícula. Após a validação dos valores, faz-se uma *query* SQL onde atualiza-se o valor do atributo, através da matrícula associada ao veículo.
- **exportarDadosParaJson**: Método para responder ao terceiro ponto do perfil Administrador, este exporta os dados de um veículo, alugueres e intervenções associados a ele para um arquivo JSON. Para tal, é realizado três *queries* SQL, uma onde se procura os dados do veículo através da sua matrícula, uma onde procura-se na vista ReservasComAluguer, se existe alugueres associados a um veículo entre duas datas. A última *query* diz respeito às intervenções, onde procura na sua tabela se existe intervenções associadas a um veículo entre duas datas. Com cada *query* executada, utiliza-se o método auxiliar gerarJsonString() para converter a informação num formato válido para um ficheiro JSON. Numa das versões deste método, a informação no formato JSON é guardada num ficheiro e na outra versão (método sem o parâmetro caminhoFicheiro) é retornada a String no formato JSON.
- **gerarJsonString**: Gera uma string no formato JSON a partir de uma matriz de resultados, um array de colunas e outros parâmetros de controlo (inicio, fim).
- **processarJson**: Método para responder ao quarto ponto do perfil Administrador, este processa uma String no formato JSON, validando e criando registos relacionados a veículos, intervenções, alugueres e reservas.
- **checkCategoria**: Verifica se uma categoria existe em uma lista de categorias.
- **checkParque**: Verifica se um parque existe em uma lista de parques.
- **reservaVeiculo**: Insere uma reserva na base de dados (após a validação dos seus parâmetros).
- **criarAluguer**: Insere um aluguer na base de dados (após a validação dos seus parâmetros).
- **lerFicheiroJson**: Lê um arquivo JSON de um caminho especificado e retorna o conteúdo como uma string.

4.3 - Cliente.

Depois, implementou-se a classe `Cliente`, responsável por todas as funcionalidades referentes ao cliente. Representa a interação entre um cliente e o sistema. Tem como atributos o seu NIF, stub (objeto auxiliar para interação com a base de dados), uma lista de unidades monetárias suportadas e por fim a tarifa.
A classe `Cliente` é constituída pelos métodos:

- **Construtor Cliente**: Inicializa a instância do cliente com o seu NIF, configura a lista de moedas suportadas e inicializa o atributo tarifa.
- **listarModelosDisponiveis:** Retorna uma matriz com as marcas e modelos de veículos disponíveis para reserva.
- **listarCategoriasVeiculosDisponiveis:** Retorna uma matriz com as categorias de veículos disponíveis.

- **TarifaModelo:** Calcula a tarifa com base no modelo do veículo e obtém a tarifa consultando a base de dados.
- **TarifaTipoVeiculo:** Calcula a tarifa com base na categoria do veículo e obtém a tarifa consultando a base de dados.
- **converterCusto:** Converte o custo final para a moeda especificada utilizando taxas de conversão predefinidas para as diferentes moedas.
- **calcularCustoPrevisto:** Calcula o custo previsto de uma reserva, levando em consideração as datas de início e fim, além do modelo ou categoria do veículo.
- **verificarDescontosDisponiveis:** Retorna uma lista de códigos de desconto válidos.
- **obterValorPercentualDesconto:** Retorna o valor percentual do desconto correspondente ao código fornecido.
- **custoComDesconto:** Calcula o custo final com o desconto aplicado.
- **clienteReservarVeiculo:** Método para responder ao primeiro ponto do perfil Cliente, este cria uma reserva para o cliente, após validar os seus parâmetros ele gera um identificador único para ser usado ao inserir os dados na reserva (ID Reserva). É executado uma *query* SQL para inserir na base de dados a reserva do cliente, caso esta tenha sucesso e o cliente tenha utilizado um desconto insere-se na tabela Desconto_Reserva uma entrada indicando o desconto e o id da Reserva.
- **clienteConsultarReservas:** Método para responder ao segundo ponto do perfil Cliente, este retorna uma matriz com informações sobre as reservas do cliente, através de uma *query* SQL sobre a vista EstadoReserva que retorna todas as suas entradas com o NIF do cliente.
- **clienteConsultarReputacao:** Método para responder a parte do terceiro ponto do perfil Cliente, este retorna a reputação do cliente, através de uma *query* SQL baseada no NIF do cliente.
- **verDescontos**: Método para responder ao terceiro ponto do perfil Cliente, este retorna os descontos disponíveis, através de uma *query* SQL baseada na data atual.
- **criarDescontoReserva:** Associa um desconto a uma reserva específica, inserindo a relação na tabela Desconto_Reserva.
- **getTarifa:** Retorna o valor da tarifa calculada.
- **getStub:** Retorna o objeto Stub associado ao cliente.

4.4 - Condutor.

Depois, implementamos a classe `Condutor`, responsável por todas as funcionalidades referentes ao condutor. Representa a interação entre um condutor e o sistema.

A classe `Condutor` é constituída pelos métodos:

- **Construtor Condutor:** Inicializa um objeto Stub para interação com a base de dados.
- **consultarReservasPendentes:** Consulta todas as reservas pendentes associadas ao condutor com o NIF fornecido.
- **validarMatriculaReservasPendentes:** Verifica se a matrícula fornecida está associada a uma reserva pendente de um condutor (através do NIF do cliente).
- **levantarVeiculo:** Método para responder ao primeiro ponto do perfil Condutor, este permite que o condutor levante um veículo previamente reservado. Para tal, após validar se o veículo selecionado pode ser levantado,

realiza três *queries* SQL para atualizar o estado do veículo de "reservado" para "alugado", trocar o estado do lugar de estacionamento de "ocupado" para "vago" e apagar a entrada da tabela LugarEstacionamento_Veiculo referente ao veículo levantado (como este vai sair do parque, não deve estar inserido nesta tabela).

- **lugaresDisponiveis:** Retorna uma lista de lugares de estacionamento disponíveis (vagos) para a morada fornecida.
- **validarLugar:** Retorna o lugar de estacionamento (especificado por piso, fila e número) se este estiver disponível na morada fornecida, se não retorna null.
- **matriculaCondutor:** Obtém a matrícula do veículo associado a um aluguer incompleto do cliente com o NIF fornecido, ou seja, indica a matrícula de um veículo com um aluguer ativo.
- **entregarVeiculo:** Método para responder ao segundo ponto do perfil Condutor, este permite que o condutor entregue um veículo alugado em um lugar de estacionamento previamente especificado. Para tal, após validar se o veículo selecionado pode ser entregue, realiza três *queries* SQL para atualizar o estado do veículo de "alugado" para "disponível", trocar o estado do lugar de estacionamento de "vago" para "ocupado" e inserir uma entrada na tabela LugarEstacionamento_Veiculo indicando o lugar do veículo num parque.
- **aluguerIdCondutor:** Obtém o ID de um aluguer associado a um condutor.
- **carrosAlugados:** Verifica se o condutor com o NIF fornecido possui veículos para entregar.
- **avaliarServico:** Método para complementar o segundo ponto do perfil Condutor, este permite que o condutor avalie o serviço associado a um aluguer. Para tal, realiza uma *query* SQL para inserir uma entrada na tabela Avaliacao indicando a avaliação e o comentário (se fornecido).
- **avaliarServicoWeb:** Semelhante ao método avaliarServico, mas para a Web.
- **isVeiculo:** Verifica se o veículo com a matrícula fornecida está no estado especificado, para tal realiza uma *query* SQL onde retorna o estado do veículo (obtido através da sua matrícula) e compara-o com o estado especificado.

4.5 - Funcionário

Posteriormente, implementamos a classe `Funcionario`, responsável por todas as funcionalidades referentes ao funcionário. Representa a interação entre um funcionário e o sistema.

A classe `Funcionario` é constituída pelos métodos:

- **Construtor Funcionario**: Inicializa um objeto Stub para interação com a base de dados.
- **consultarReservasPendentes**: Retorna reservas que ainda não foram vinculadas a um aluguer.
- **consultarReservasPendentesWeb**: Semelhante ao anterior, mas para a Web.
- **verificarReservaPendente**: Verifica se uma reserva com um ID específico está pendente.
- **consultarVeiculosDisponiveis**: Retorna a matrícula, marca e modelo de veículos disponíveis que correspondem às características de uma reserva específica (categoria ou modelo).
- **funcionarioCriarAluguer**: Método para responder ao primeiro ponto do perfil Funcionário, este cria um aluguer para uma reserva. Gera um ID exclusivo para o aluguer, verifica a disponibilidade da matrícula e o estado da reserva antes de realizar a operação. Para realizar a operação, faz-se apenas uma *query* SQL onde

insere-se na tabela do Aluguer uma nova entrada, com o identificador de aluguer criado, a matrícula do veículo para atribuí-lo ao aluguer e o identificador da reserva.

- **alterarEstadoVeiculo**: Método que complementa o primeiro ponto do Funcionário, este atualiza o estado de um veículo para "reservado" após ser atribuído a um aluguer.
- **gerarIdAluguer**: Gera IDs exclusivos para novos alugueres.
- **funcionarioLocalizarVeiculo**: Método para responder ao segundo ponto do perfil Funcionário, este retorna a localização de um veículo. Para tal, realiza-se uma *query* SQL, onde procura-se na tabela LugarEstacionamento_Veiculo a localização de um veículo pela sua matrícula.
- **funcionarioLocalizarVeiculoWeb**: Semelhante ao método anterior, mas retorna uma String para ser utilizada na versão Web.
- **funcionarioCriarIntervencao**: Método para responder ao terceiro ponto do perfil Funcionário, este regista uma intervenção para um veículo, após validar que o custo é positivo. Para tal, realiza-se uma *query* SQL, onde insere-se na tabela Intervencao uma nova entrada.
- **funcionarioProcurarCliente**: Método para responder ao quarto ponto do perfil Funcionário, este procura informações sobre um cliente pelo nome.
- **funcionarioAutoCompleteCliente**: Método para complementar o quarto ponto do perfil Funcionário, este retorna uma lista de todos os nomes dos clientes, este método é utilizado na servlet FuncionarioAutocomplete, para realizar a funcionalidade desejada neste ponto.
- **getData**: Método para complementar o quarto ponto do perfil Funcionário, este filtra os nomes para serem colocados como sugestões para o *autocomplete*.
- **alugueresAFinalizar**: Retorna uma lista de alugueres que ainda não foram finalizados.
- **alugueresAFinalizarWeb**: Semelhante ao anterior, mas para a Web.
- **verDetalhesAluguerCondutor**: Retorna os detalhes de um aluguer, através de uma query SQL que consulta a vista DetalhesAlugueresCondutor, através do ID de um Aluguer.
- **funcionarioFinalizarAluguer**: Método para responder ao quinto ponto do perfil Funcionário, este finaliza um aluguer, atualizando informações como custo final (com desconto aplicado), avaliação do condutor, e quilometragem percorrida
- **atualizarReputacaoCondutor**: Calcula e atualiza a reputação de um condutor com base na média das avaliações dos alugueres que ele está associado.
- **atualizarQuilometragemVeiculo**: Atualiza a quilometragem total de um veículo com base em uma matrícula fornecida, somando o valor informado.
- **funcionarioProcurarCondutorDeUmVeiculo**: Método para responder ao sexto ponto do perfil Funcionário, este procura o condutor de um veículo específico em uma data e hora fornecida.
- **verificarDisponibilidadeLugar**: Verifica se um lugar de estacionamento específico está no estado “vago”.
- **atualizarLugarEstacionamento**: Atualiza o estado de um lugar de estacionamento para ocupado e insere uma entrada na tabela LugarEstacionamento_Veiculo com as informações da matrícula de um veículo e com os dados do lugar de estacionamento.
- **vagoLugarEstacionamento**: Libera o lugar de estacionamento atualmente associado a um veículo (muda o seu estado para “vago”).

4.6 - Gerente

Por fim, implementamos a classe `Gerente`, responsável por todas as funcionalidades referentes ao gerente. Representa a interação entre um gerente e o sistema.

A classe Gerente é constituída pelos métodos:

- **Construtor Gerente**: Inicializa um objeto Stub para interação com a base de dados.
- **gerenteHistoricoVeiculo**: Método para responder ao primeiro ponto do perfil Gerente, este retorna o histórico completo ou de apenas alguns meses de intervenções e alugueres de um veículo específicado pela sua matrícula.
- **gerenteRankingMenorLucro**: Método para responder ao segundo ponto do perfil Gerente, este retorna o *ranking* das marcas de veículos que geraram o menor lucro. Para tal, realiza-se uma *query* SQL para consultar a vista LucroTotalPorMarca.
- **gerenteRankingMelhorAvaliacao**: Método para responder ao terceiro ponto do perfil Gerente, este retorna o *ranking* dos modelos de veículos com as melhores avaliações em um período específico (número de semanas). Para tal, realiza-se uma *query* SQL onde faz-se a média das avaliações para os alugueres dentro do período de tempo.
- **gerenteRankingMenoskm**: Método para responder ao quarto ponto do perfil Gerente, este retorna o *ranking* dos veículos que percorreram menos quilômetros nos últimos três meses. Para tal, realiza-se uma *query* SQL onde soma-se os quilômetros percorridos por veículo (através dos alugueres) dentro do período de tempo.
- **gerenteRankingClientes**: Método para responder ao quinto ponto do perfil Gerente, este retorna o *ranking* dos clientes com melhor reputação em uma determinada freguesia.
- **listarFreguesias**: Lista todas as freguesias distintas presentes no sistema.
- **listarFreguesiasWeb**: Semelhante ao método anterior, mas gera uma String com as opções de um `select` HTML.

4.7 - Web

Na parte web do projeto, cada tipo de utilizador representa uma entrada do menu principal, ao selecionar uma entrada o utilizador depara-se com o menu respetivo a esse tipo de utilizador, nestes menus cada requisito funcional definido (correspondente a uma alínea do enunciado) foi representado como uma entrada nestes menus.
Cada funcionalidade é gerida por uma ou mais páginas JSP (JavaServer Pages), que contêm formulários específicos de acordo com as necessidades de interação do utilizador. As páginas JSP são responsáveis por permitir o preenchimento dos formulários e enviar os dados coletados para uma Servlet correspondente.

Após receberem os dados submetidos pelos formulários das páginas JSP, as Servlets, instanciam as classes necessárias, validam os dados através de métodos específicos dessas classes e executam as funções apropriadas para atender ao requisito funcional associado. Esta estrutura garante uma separação clara entre a camada de apresentação (JSP) e a camada de processamento (Servlets).

Para funcionalidades que envolvem a atualização ou inserção de novos registos na base de dados, as Servlets processam a operação e enviam ao utilizador mensagens indicando se a operação foi concluída com sucesso ou se

ocorreu algum erro. Esta abordagem contribui para uma experiência de utilização mais eficiente e interativa, fornecendo *feedback* imediato sobre as ações realizadas.

Conclusão

O desenvolvimento deste projeto proporcionou uma experiência completa no desenvolvimento de aplicações web em conjunto com bases de dados na linguagem SQL, consolidando os conhecimentos adquiridos na unidade curricular de Sistemas de Bases de Dados. A divisão do projeto em duas partes complementares demonstrou que a implementação prática é essencial para validar e ajustar decisões tomadas na fase inicial. Na segunda parte do projeto, percebemos que algumas soluções concebidas na primeira parte do projeto não funcionavam da melhor forma e não eram ideais para a resolução de certos problemas. A implementação da aplicação web ajudou-nos a identificar e corrigir erros na estrutura e nos processos da nossa base de dados, permitindo criar uma solução mais eficiente e alinhada às necessidades reais do sistema.

Na primeira parte, foi possível aprofundar as competências em modelagem de dados, desde a conceção inicial até à implementação física da base de dados, utilizando a linguagem SQL como ferramenta principal. Este processo garantiu uma estrutura sólida capaz de atender às necessidades de uma empresa de aluguer de veículos.

Na segunda parte, a criação de uma aplicação web funcional e interativa destacou a importância de integrar a base de dados com tecnologias como JDBC, JSP e Servlets. Essa integração permitiu a manipulação dinâmica dos dados, criando uma interface acessível e intuitiva para os utilizadores finais, seja para adicionar novos registos, modificar informações existentes ou consultar dados relevantes.

O projeto como um todo demonstrou a importância de uma abordagem estruturada desde o começo, com vista em facilitar os processos futuros. Além disso, enfrentaram-se desafios na parte da implementação da aplicação web que acabaram por melhorar as nossas capacidades em modelar bases de dados mais eficientes e estruturadas desde o momento da sua conceção. O grupo acabou por não conseguir implementar o sistema de fotos e vídeos relacionados com os veículos. No entanto acreditamos que para tal implementação seria necessário criar uma nova entidade 'ObjetoMultimedia' que representaria as fotos e vídeos relacionados aos veículos.

Em conclusão, o projeto atingiu quase todos os objetivos propostos, oferecendo uma solução para os processos de uma empresa de aluguer de veículos.

Bibliografia

Consulta:

Moodle: Trigo, P., Filipe, P., & Teófilo, A. (2024 - 2025). Sistemas de Bases de Dados